Edge cases (context limit) — Hints

Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and forcing it on purpose
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and forcing it on purpose

Each model has a fixed token budget for the whole request (system + user + history + the reply it's about to generate). Send a request whose token count exceeds that budget, and the API raises an error rather than silently truncating anything on your behalf — truncating your input for you would be a dangerous, silent behavior for an API to have, since it could quietly drop something important without telling you.

`huge_input = "word " * 200_000` builds a string far larger than any current model's context window in a single line. Wrap the API call in `try: ... except openai.BadRequestError as e:` — a context-length problem surfaces as a `BadRequestError` (HTTP 400), not a special "context" exception of its own, so catching that specific type (not a bare `except:`) is what actually shows you which real exception class this is. Print `type(e)` and `str(e)` — the message text usually names the token counts involved directly, which is worth reading once for real.

Hint 2 — The plan, and almost the whole thing

```
import openai
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(type(e).__name__)
    print(str(e))
```

Run it — then compare catching the *specific* error type against the bare `Exception` from Basic. Notice `openai.BadRequestError` is what you'd actually write in real code, since a bare `except` would also silently swallow a genuine bug in your own request-building code, not just this one expected failure. Then compare against the Solution.

Full solution: Show me the solution